Uniform introduction on raw terms

A smaller presentation of intuitionistic primitive recursive arithmetic

Research prepared for Vladimir Reshetnikov

November 12, 2026

Abstract

We simplify the latest raw guarded presentation in ConciseIPRA by deleting its exceptional rule for recognizing a naming formula. Introduction always uses the newest numerical parameter, and uses zero when the ambient numerical scope is empty. There is no supplied witness, witness-mode bit, search, naming-pattern test, or computation normalization. We prove equality of all open guarded consequence relations with the unrestricted-witness calculus. Consequently the ordinary realization remains equiconsistent and bi-interpretable with full first-order intuitionistic primitive recursive arithmetic; both numerical domains are whole, equality is actual equality, and both numerical comparison graphs are the identity.

The proof closes a bootstrap dependency. It first embeds the purely equational iterator calculus using regenerated proof families, then constructs predecessor without numerical coverage, and only afterwards derives arbitrary names. The inherited scalar predecessor construction is retained; an equational interpolation lemma removes its apparent reliance on Boolean case distinctions. A tightly matched, complete Python checker shrinks from 3,688 to 3,439 normalized bytes, from 1,159 to 1,073 significant tokens, and from 1,339 to 1,248 AST nodes. This is a proved local simplification, not a claim of an absolute smallest axiomatization or a size record over all certificate formats in the repository.

1 What is new, and what is being compared

The comparison snapshot is ProveIt commit

`9f9545f5ffd9e0392d4a36639503aca12383450a.`

The inspected directory is `ConciseIPRA` [1]. Its main README describes 79 filed research packages. In addition, the current `docs/incoming` directory contains five archives. We retrieved and inspected those archives as well. This matters: two already eliminate supplied introduction witnesses. The result below is compared with that newer work, rather than with an obsolete witness-taking kernel.

The incoming `parameter_intro` package already uses the newest parameter, or zero, with full computation normalization and a counter-aware successor-seeded recursor [3]. We do not claim that a fixed newest witness is a new idea by itself. The incoming `deterministic_introduction` package works on raw scalar iterator terms [2]. Its introduction chooses a witness by the rule

$$w(B,\Pi)=\begin{cases} t, & B \text{ is literally } C(y,0,x)=C(y,0,t),\ t \text{ outer-scoped},\\ \text{newest parameter in } \Pi, & \Pi\neq\varnothing,\\ 0, & \Pi=\varnothing. \end{cases}$$

Our change deletes the first line and its entire recognizer. The remaining rule is sufficient on the same raw terms and the same guarded formulas. Every other instruction in the selected checker is unchanged.

This distinction is the concrete simplification. It removes a mathematical exception and its implementation, while retaining the existing arithmetic basis and avoiding a normalization layer. It also establishes that the exception was not needed to bootstrap arbitrary names.

Complete acceptance module	Bytes	Tokens	AST nodes
Incoming raw deterministic introduction	3688	1159	1339
Uniform introduction, this package	3439	1073	1248
Reduction	249	86	91

Table 1: Matched complete programs, including input handling, syntax, scope checks, all primitive rules, and the accept/reject wrapper.

The repository also has smaller programs using other interfaces. For example, the incoming raw evaluation-discharge checker reports 1,892 bytes, its zero-normal companion 1,858 bytes, and the full-normal parameter-introduction checker 1,925 bytes [4, 3]. Compact public-goal codecs are smaller still. Thus Table 1 does not establish a corpus-wide size minimum. The result is a strict reduction of the current raw deterministic axiomatization, with a matched improvement in all three stated source-size coordinates.

2 The complete axiomatization

2.1 Terms, formulas, and scopes

There is one numerical sort. The finite raw terms are

$$t ::= x \mid 0 \mid St \mid I(n, a; u.h).$$

Only u is bound in h. The counter n, initializer a, and all other parameters of h are outside this binder. Substitution is capture avoiding. No computation equation is built into term equality or substitution. Define the abbreviation

$$C(v, a, b) := I(v, a; u.b), \qquad 1 := S0,$$

where the displayed state variable is absent from b.

A formula is an equation $s = t$ or a paired formula $D_{x,y}B$. There is one syntactic restriction: if B is an equation immediately under D, it must literally have the form

$$I(y, 0; u.h) = I(y, 0; v.k). \tag{Guard}$$

The updates h, k may depend on their state variables, on x, y, and on outer parameters. If B is compound, its root is another D, and its own innermost equation must satisfy the same restriction. An equation at the root of a formula is unrestricted. The designated contradiction is $0 = S0$.

A numerical scope is an ordered list $\Pi = (a_1, \ldots, a_n)$ of distinct parameters. Define

$$\boxed{\omega(\Pi) = a_n \text{ if } n > 0, \qquad \omega(\varnothing) = 0.} \tag{1}$$

The ordering is proof syntax; it imposes no numerical ordering on elements of a model. Every rule specifies which parameters are added and when.

2.2 Contexts and ordinary assumptions

A context is a finite reusable list of clauses $\langle A; y.B(y)\rangle$, with y absent from A. Both components are canonical formulas in the appropriate scopes. Its ordinary reading is $A \to \forall y B(y)$. Clause indices are syntactic addresses, not object-level numbers.

Set $H = (I(0,0;u.0) = 0)$. An ordinary assumption F abbreviates the constant clause $\langle H; v.F\rangle$, with v absent from F. The formula H is an instance of the first computation schema, not an extra axiom. Consequently an ordinary assumption is retrieved by Use and one computation proof. A closed proof begins in empty numerical scope and with an empty clause context.

Write $\Pi; \Gamma \vdash_{\mathcal{G}_n} A$ for derivability. In displays where the numerical scope is clear, we suppress Π.

2.3 Two arithmetic schemas and four inference rules

The complete nonlogical computation schemas, in every context, are

$$I(0, a; u.h) = a, \tag{I0}$$

$$I(Sn, a; u.h) = h[I(n, a; u.h)/u]. \tag{IS}$$

They range over all well-scoped finite terms of the grammar above. The remaining primitive rules are exactly the following.

Use. For $\langle A; y.B\rangle \in \Gamma$ and any well-scoped term t,

$$\frac{\Gamma \vdash A}{\Gamma \vdash B(t)}. \tag{Use}$$

Transport. For a canonical one-parameter formula template $F(v)$,

$$\frac{\Gamma \vdash s = t \qquad \Gamma \vdash F(s)}{\Gamma \vdash F(t)}. \tag{Tr}$$

The placeholder is a numerical variable. No substitution of proofs or test of semantic equality occurs in this rule.

Open.

$$\frac{\Pi; \Gamma \vdash D_{x,y}B(x,y) \qquad \Pi, b; \Gamma, \langle B(b,0); y.B(b,y)\rangle \vdash A}{\Pi; \Gamma \vdash A}. \tag{Open}$$

Here b is fresh for Π, Γ, A and the fixed parameters of B. The branch is checked in the scope with b appended; the conclusion is checked in the old scope, so b cannot escape, including inside an unused iterator body.

Uniform introduction. Put $w = \omega(\Pi)$ *before* adding the step parameter. Let

$$\eta_B(z) = \begin{cases} z, & B \text{ is an equation,} \\ 0, & B \text{ is compound.} \end{cases}$$

Then

$$\boxed{\frac{\Pi, z; \Gamma, B(w, \eta_B(z)) \vdash B(w, Sz)}{\Pi; \Gamma \vdash D_{x,y}B(x,y)}}. \tag{Intro}$$

The parameter z is fresh for the old scope, context, and fixed matrix parameters. At an equation this is a progress step; at a compound matrix it is a zero/successor case step. The matrix must satisfy Guard when it is an equation. Introduction receives no witness data.

Remark 2.1 (Exhaustiveness). There are no further primitive logical or nonlogical axioms, successor axioms, equality axioms, induction schemas, or inference rules. In particular, reflexivity, ordinary hypothesis use, structural rules, and the usual intuitionistic connectives are derived. The checker has six instruction tags: the two computations and the four rules above.

Let $\mathcal{G}$ denote exactly this calculus with an arbitrary well-scoped witness t in Intro. Let $\mathcal{G}_{\mathrm{d}}$ denote the incoming calculus with its literal-name exception. The inclusions

$$\mathcal{G}_{\mathrm{n}} \subseteq \mathcal{G} \qquad \text{and} \qquad \mathcal{G}_{\mathrm{d}} \subseteq \mathcal{G}$$

are immediate on derivations. The substantial direction is completeness of $\mathcal{G}_{\mathrm{n}}$.

3 The equivalence theorem and its proof discipline

Theorem 3.1 (Exact open-consequence equivalence)**.** *For every ordered numerical scope* Π*, every well-scoped canonical clause context* Γ*, and every canonical formula* A*,*

$$\Pi;\Gamma \vdash_{\mathcal{G}_{\mathrm{n}}} A \quad\Longleftrightarrow\quad \Pi;\Gamma \vdash_{\mathcal{G}} A \quad\Longleftrightarrow\quad \Pi;\Gamma \vdash_{\mathcal{G}_{\mathrm{d}}} A.$$

The translations are effective transformations of finite derivations; the terms, formulas, and external assumptions are unchanged.

The easy directions use the respective witnesses as unrestricted witnesses. The difficult direction is proved in Sections 4–6. It has the following dependency order:

1. Establish replacement by uniformly regenerated proof families.
2. Embed the purely equational iterator fragment, before any names.
3. Obtain predecessor in that fragment, without numerical coverage.
4. Derive arbitrary names using the newest parameter.
5. Regenerate source derivations under arbitrary receiving scopes, recovering arbitrary-witness introduction.

There is a real issue here. Adding a numerical parameter changes $\omega(\Pi)$. Replaying a proof unchanged in a larger scope is generally invalid. The argument never assumes that replay is a structural rule. Likewise, it never substitutes an arbitrary term into a completed restricted target proof. Where a source step needs a new instance, its proof is regenerated in that instance.

4 Hereditary proofs and the arithmetic bootstrap

Definition 4.1 (Hereditary derivability)**.** Write $\Gamma \Vdash A$ if, for every injective renaming of its displayed parameters into a finite receiving numerical scope and every receiving clause context containing the renamed Γ, there is an effectively generated $\mathcal{G}_{\mathrm{n}}$ proof of the renamed A. The generation may use the receiving scope and context as inputs.

This is a metatheoretic property of a family of finite proofs. It is not an additional rule or oracle in the checker. Identity renaming and the unchanged context show that hereditary derivability implies ordinary derivability. Context membership and all the algebraic rules have hereditary proofs.

Lemma 4.2 (Replacement with hereditary donors)**.** *Suppose* $\Gamma \Vdash A$ *and* $\Pi; \Delta, A \vdash_{\mathcal{G}_n} B$*, where* Δ *contains* Γ *in the displayed scope and* A *is added as an ordinary assumption. Then* $\Pi; \Delta \vdash_{\mathcal{G}_n} B$*. The same statement holds under renamed embeddings of* Γ*. Replacement is valid below numerical binders; all donor hypotheses must remain available.*

Proof. Induct on the receiving derivation. Preserve the exact numerical scope at each of its nodes. At a Use of the constant clause for the removed assumption, generate the donor proof in the numerical scope and context of that leaf. All other Use addresses are adjusted for the deleted clause. All other rule nodes retain their own scopes and parameters; in particular their selected newest witness is unchanged. Below Open and Intro the donor is generated in the extended scope, rather than obtained by weakening an already generated proof. The recursion is on a finite receiving proof and proves the assertion without a target numerical weakening theorem. □

4.1 Padding and cases without names

For an equation $E = (s = t)$ put

$$G_y E = (C(y, 0, s) = C(y, 0, t)), \qquad L(E) = D_{a,v} G_v E,$$

where a, v are fresh for E. Both are abbreviations in the raw syntax. The coordinate a is unused. Computation and transport give $G_0 E$ and $G_{Sz} E \dashv\vdash E$. Therefore

$$L(E) \dashv\vdash E \tag{2}$$

uniformly in every receiving scope and additional context. For the forward introduction of $L(E)$, the retained E proves its successor row, independently of the progress assumption and of the chosen witness. For the reverse direction, open $L(E)$, prove its zero row, and Use at 1.

For equations $U = (s = t)$ and $V = (r = q)$ define

$$U \Rightarrow_g V = D_{a,w} L\big(C(w, s, r) = C(w, t, q)\big). \tag{3}$$

Its zero endpoint is interderivable with U, and every successor endpoint with V. The outer witness coordinate is unused. It follows from Intro, Open, Use, (2), and Lemma 4.2 that (3) has the ordinary introduction and elimination rules for an implication between equations, in hereditary proof families.

Numerical cases are likewise available in hereditary families. For a compound $A(y)$, introduce $D_{a,y} A(y)$ with unused a, ignoring its base assumption in the supplied successor branch; open it and use the given base at the desired target. For an equation $E(y)$ apply the compound construction to $L(E(y))$ and use (2). These constructions do not require any name theorem or chosen existential witness.

4.2 Embedding a source equational calculus

Let $\mathcal{E}$ be ordinary equational reasoning on $0, S, I$, with the two computation equations and contextual induction for one literal equation:

$$\frac{\Gamma \vdash E(0) \qquad \Gamma, E(z) \vdash E(Sz)}{\Gamma \vdash E(t)}, \qquad z \notin \mathrm{FV}(\Gamma, t). \tag{EI}$$

The eigenparameter z is also absent from the fixed parameters of $E(y)$. The source calculus has no coverage, separation, or non-equational induction axiom. Its assumptions are equations. Equality rules include congruence and substitution in finite term templates.

Lemma 4.3 (Equational embedding before names)**.** *For every finite $\mathcal{E}$ derivation $\Gamma \vdash E$ and every numerical term substitution σ, its instance has a hereditary target proof:*

$$\Gamma\sigma \Vdash E\sigma.$$

Proof. Induct on the source derivation, simultaneously for all substitutions and receiving contexts. Computation and equality use the hereditary algebraic rules and Lemma 4.2. Here reflexivity itself is derived: if $r = I(0, t; u.0)$, use the equation $r = t$ as both Transport premises with template $v = t$ to obtain $t = t$.

Consider (EI), after substitution of its fixed parameters; write its invariant as $E(y)$ and let $J(y) = G_y E(y)$. By the source induction hypothesis, the proper step subderivation can be regenerated at $y = 0$ and at $y = Sw$, in arbitrary receiving scopes. Thus, with the fixed outer context retained, there are hereditary proofs $E(0) \vdash E(1)$ and $E(Sw) \vdash E(SSw)$. This does not substitute into a completed target proof.

Put $\Phi(y) = J(y) \Rightarrow_g E(Sy)$. At $y = 0$, its implication proof ignores $J(0)$ and uses the source base and regenerated zero step. At $y = Sw$, remove the successor guard from $J(Sw)$ and use the regenerated successor step. Hereditary numerical cases yield $\Phi(z)$ under the outer context, which is independent of z. Under $J(z)$, implication elimination yields $E(Sz)$ and hence $J(Sz)$. Guarded progress introduction with unused witness coordinate now proves $D_{a,y}J(y)$. Open it; its zero row is a computation theorem. Use at Sw and remove the successor guard to get $E(Sw)$. Numerical cases with the original base give $E(t)$.

All recursive proof-generation calls in this argument concern proper subderivations of the source proof. Every target replacement has an already hereditary donor. This establishes a well-founded construction and completes the induction. □

Proposition 4.4 (Predecessor before names)**.** *There is a fixed finite raw scalar term P such that, hereditarily and under all term substitutions,*

$$P(0) = 0, \qquad P(St) = t.$$

In particular $Sb = St$ implies $b = t$ by equality reasoning.

Proof. Appendix A constructs P in $\mathcal{E}$, using only computation, equality, and equation induction. Apply Lemma 4.3. The successor-injectivity conclusion follows by applying P to the assumed equation and using the two instantiated predecessor equations. □

The equational construction is important. Using the previously established whole iPRA equivalence to obtain predecessor here would make the naming bootstrap circular. Appendix A instead removes the Boolean case distinctions from the inherited scalar construction by an explicit equation-induction argument.

5 Uniform naming with no syntactic exception

Define the canonical packet

$$A(r) := D_{x,v}(C(v, 0, C(r, 0, Sx)) = C(v, 0, r)). \tag{4}$$

Both sides of its atomic matrix are iterators with counter v and initializer zero, so (4) satisfies Guard. Its ordinary meaning is

$$A(r)^\circ \iff \exists x(C(r,0,Sx) = r). \tag{5}$$

The equivalence follows from its computationally true zero row, its row at 1, and numerical coverage in the ordinary interpretation. Equation (5) explains the packet; the syntactic proof below does not assume coverage.

Lemma 5.1 (Uniform coverage packet)**.** *For unused a, $D_{a,r}A(r)$ has a hereditary $\mathcal{G}_{\mathrm{n}}$ proof. In every receiving scope and context, $A(St)$ has an effectively generated proof for every well-scoped term t.*

Proof. First, $A(0)$ can be introduced at whatever witness (1) selects. Its successor row computes to $0 = 0$, independently of that witness and of its local progress assumption.

To introduce $D_{a,r}A(r)$, its matrix is compound, so Intro asks for $A(Sz)$ under its base assumption, with a newly allocated newest parameter z. Introduce $A(Sz)$ in that branch. The parent scope of this inner introduction ends in z, so its witness is exactly z. The successor row computes to $Sz = Sz$ and does not use its progress assumption. The outer base assumption can be ignored. This builds the whole outer proof in any receiving scope.

The whole construction is hereditary because it allocates z anew on each regeneration. We do not claim that its isolated internal proof of $A(Sz)$ can be replayed after z ceases to be newest.

Open $D_{a,r}A(r)$. The dummy witness is absent from the clause $\langle A(0); r.A(r)\rangle$. Regenerate the first construction of $A(0)$ in that enlarged scope and Use the clause at St. The result $A(St)$ contains no dummy witness, so Open discharges it. □

Lemma 5.2 (Alias extraction)**.** *For any outer term t, open $A(St)$ at a fresh newest parameter b. In that opened context, $b = t$ has a proof that can be regenerated in every later extension of the numerical scope.*

Proof. The opened clause has a zero row that is a computation theorem. Use it at 1. Two successor computations give $Sb = St$. Apply Proposition 4.4. The retained clause and the hereditary predecessor instances permit this extraction in any later scope, including below a new step parameter. It is unnecessary to weaken a previously completed extraction proof. □

Corollary 5.3 (The old literal naming theorem is derived)**.** *For every term t, the formula*

$$D_{x,y}(C(y,0,x) = C(y,0,t)) \tag{6}$$

has a hereditary $\mathcal{G}_{\mathrm{n}}$ proof, with no pattern-based witness selection.

Proof. Generate $A(St)$ and open it at newest b. Extract $b = t$. Introduce (6); its selected witness is now b. In its step branch, regenerate the alias equality and use the selector computations to prove the successor row, ignoring the progress hypothesis. Discharge the packet opening. Every fresh witness is regenerated inside the construction, so the resulting proof family is hereditary. □

6 Completeness and both interpretations

6.1 Recovering every unrestricted introduction

We prove the nontrivial implication of Theorem 3.1 in a strong form: every $\mathcal{G}$ derivation gives a hereditary $\mathcal{G}_{\mathrm{n}}$ proof of the same sequent. Induct on the source derivation, universally over injective

parameter embeddings and receiving contexts. Computations, Use, and Transport are copied with their parameters. For Open, allocate a new receiving parameter and apply the induction hypothesis to the proper branch derivation with that extended embedding.

Consider a source introduction at an arbitrary term t, with matrix B and step derivation

$$\Gamma, B(t, \eta_B(z)) \vdash_{\mathcal{G}} B(t, Sz). \tag{7}$$

In the receiving scope, generate $A(St)$ and open it at a new newest b. Introduce the desired $D_{x,y}B(x, y)$ there. This introduction selects b, and allocates its new step parameter z only afterwards. In that branch, Lemma 5.2 regenerates $b = t$. Transport turns the local assumption $B(b, \eta_B(z))$ into $B(t, \eta_B(z))$. The source induction hypothesis translates the proper derivation (7) under the enlarged parameter embedding and context. Lemma 4.2 supplies its required ordinary assumption from the hereditary transported assumption. The result is $B(t, Sz)$; transport back along the alias gives $B(b, Sz)$.

Close the deterministic introduction and the alias opening. The goal has no b, so the eigenparameter condition holds. The two modes agree because η_B depends on the matrix root, which the parameter embedding and witness substitution do not change. All recursive calls are to proper source subderivations; the packet and alias proofs were constructed independently beforehand. This proves $\mathcal{G} \subseteq \mathcal{G}_{\mathrm{n}}$ and finishes Theorem 3.1, using the incoming equivalence $\mathcal{G}_{\mathrm{d}} = \mathcal{G}$ [2].

Remark 6.1 (Structural rules after completeness). The theorem now gives ordinary weakening, injective renaming, cut, and numerical substitution as admissible consequence transformations, by translation through $\mathcal{G}$. It does not say that an unchanged proof tree passes the checker after a numerical scope extension.

6.2 Ordinary realization and full iPRA

The ordinary reading is

$$(s = t)^{\circ} = (s = t), \qquad (D_{x,y}B)^{\circ} = \exists x\big(B^{\circ}(x, 0) \to \forall y\, B^{\circ}(x, y)\big). \tag{8}$$

Clause readings are conditional universal statements as above. The domain formula is $x = x$, and numerical equality is actual equality. The guarded zero row makes the atomic introduction in (8) sound by equation induction; the compound introduction is sound by numerical zero/successor coverage. Open, Use, and Transport have their usual intuitionistic readings.

For clarity, the inherited ordinary arithmetic theory T_I is intuitionistic first-order logic with equality, scalar iterators with their two equations, contextual induction for literal equations, and

$$\forall x\,(x = 0 \vee \exists y\, x = Sy), \qquad (0 = S0) \to \bot. \tag{9}$$

The falsity bridge in (9) is explicit. A native presentation with designated contradiction $0 = S0$ must not silently claim that positive equations alone exclude the singleton model.

The existing guarded translation from the unrestricted Choice-DI calculus pads equation matrices. Let $P_{\mathrm{fm}}(E) = L(E)$ at equations and $P_{\mathrm{fm}}(A) = A$ at compound formulas. Its formula translation is

$$T(s = t) = s = t, \qquad T(D_{x,y}B) = D_{x,y}P_{\mathrm{fm}}(TB). \tag{10}$$

Literal inclusion is the reverse translation. The inherited padding and binder-congruence proofs give both formula round trips uniformly under assumptions [5]. Theorem 3.1 transfers these very same results to $\mathcal{G}_{\mathrm{n}}$; it is equality of open consequence relations, not merely agreement on closed arithmetic sentences.

The remaining inherited arithmetic bridge constructs a genuine predecessor, equality reflection, quantifier-free induction, pairing, and a compiler for every fixed primitive-recursive description in that order [6, 7]. In one direction, interpret a scalar iterator by primitive recursion with an update that ignores the counter. In the other, if π, L_0, L_1 are the constructed pairing and projections, the recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p})$$

is compiled through

$$\begin{aligned} V(n,\vec{p}) &= I(n, \pi(0, g^*(\vec{p})); u.\pi(SL_0(u), h^*(L_0(u), L_1(u), \vec{p}))), \\ f^*(n,\vec{p}) &= L_1(V(n,\vec{p})). \end{aligned} \tag{11}$$

This is external recursion on a finite function description; there is no universal total evaluator quantified over all primitive-recursive codes. Equation induction establishes the compiled defining equations.

For both interpretation composites, internal uniqueness of iteration and primitive recursion identifies the double-translated operations with the originals. The same induction over finite formula syntax, using (8), (10), and the ordinary logical macros, identifies every formula with its round trip. No quotient or restriction to standard numerals occurs.

Theorem 6.2 (Bi-interpretability and equiconsistency)**.** *The ordinary realization presented by* $\mathcal{G}_{\mathrm{n}}$ *and full first-order* iPRA *are bi-interpretable. In both directions the numerical domain is whole, equality is actual equality, and the graph of the comparison isomorphism is* $x = y$*. Both formula composites are provably equivalent to the original formulas. Effective translations of finite proofs preserve contradiction in both directions, so the theories are equiconsistent.*

Proof. Compose Theorem 3.1, the guarded/Choice-DI translations and their formula round trips, and the native/T_I/iPRA bridge. The first equivalence is identity on syntax and numerical domain; the inherited interpretations have the properties just described. Their composite therefore has those properties as well. The explicit falsity bridge and designated contradiction give the consistency equivalence. This is an external metatheorem, with no claim that either theory proves its own consistency. □

7 Executable presentation and verification

7.1 Certificate grammar

The executable checker retains the incoming strict format. The JSON object has exactly the keys `goal` and `proof`. Numerical syntax uses integer zero, positive de Bruijn indices for bound variables, and negative levels for free parameters. Successor is `[0,t]`, iteration is `[n,a,h]`, an equation is `[s,t]`, and a paired formula is `[B]`. The scopes and the sorts distinguish the two uses of binary lists. The iterator body binds one variable; the paired matrix binds two.

The full instruction inventory is

Instruction	Contents
`zero`	initializer, update
`succ`	counter, initializer, update
`use`	clause index, target term, antecedent proof
`tr`	formula template, equation proof, instance proof
`open`	paired major proof, branch proof
`intro`	matrix, branch proof

Every constructed output is checked for sort, guard, and scope. Inputs use exact integer types, so Boolean values are rejected as numerical syntax. A branch cannot export its local parameter even in an unused iterator update. Python's optimization flag does not disable any acceptance-critical check; the program does not use `assert` for input validation.

The sole change to the incoming checker is

```
-        t = witness(matrix, scope)
+        t = -scope
```

together with deletion of the now unused `witness` function. At scope zero, `-scope` is literal zero; otherwise it names the newest parent parameter. The fresh step parameter remains `-scope - 1`. No part of the new proof construction is hidden inside the verifier.

7.2 Measurements and limits

The measurement script removes actual Python docstrings, applies `ast.unparse`, and adds one terminal newline. Bytes are UTF-8 bytes of this normalized complete module. AST size counts all nodes from `ast.walk`. Significant tokens exclude encoding, end marker, indentation, line separators, and comments by their named `tokenize` constants. The results reproduce Table 1 on CPython 3.12.14.

The reductions are 6.75% in bytes, 7.42% in tokens, and 6.80% in AST nodes. The comparison programs have identical parser, public language, error handling, computation rules, transport rule, and scope discipline. This is an axiomatization change with a measured implementation benefit, rather than a serialization-only change.

The supplied validation scripts run 11 accepted and 17 rejected certificates in both normal and optimized Python: 56 CLI checks, all passing. They exercise accepted derivations, malformed syntax, guards and binders, eigenparameter escape, the whole universal coverage construction, and the exact syntactic patch against the frozen source control. A separate comparison confirms that the two witness policies need different branch certificates even though their theorem sets coincide. All eleven positive certificates are also elaborated by inserting their selected witnesses and replayed successfully in the frozen unrestricted source checker. Exact receipts are generated by `reproduce.py`; they distinguish executed finite checks from the universal written proofs in this paper.

The implementation does not include a complete executable compiler from every unrestricted proof to the new calculus. That transformation is proved in Sections 4–6. In particular, the full predecessor derivation and the general hereditary-proof metatheorem have not been checked by Lean or Rocq in this package. Existing upstream Lean theorems are credited foundations, not a claim that the newly restricted calculus is already formalized. Large finite certificates can also exceed a particular Python process's stack or memory limits; no uniform resource bound is asserted.

8 What this settles and what remains open

The current raw deterministic presentation does not require a special case for literal naming matrices. Arbitrary names can be generated using its ordinary numerical parameter supply. This removes a branch from the mathematical specification and 249 normalized bytes from its complete strict checker.

The result also clarifies two dependencies. The inherited scalar predecessor construction can be carried out in the purely equational fragment; numerical coverage is not necessary for its two defining equations. Conversely, a proof that uses the newest parameter is not automatically stable

when another parameter is appended. Hereditary regeneration is the reason that the naming and completeness arguments remain valid.

An absolute smallest presentation is not established. It would require a fixed comparison class for languages, syntax checking, certificate protocols, libraries, and source-size conventions, together with a lower-bound argument. The compact codecs and the newer three-family calculi remain relevant alternative implementations. The present contribution is the precise deletion and equivalence theorem above, with its complete, reproducible checker comparison.

A Predecessor without numerical coverage

Let $\mathcal{E}$ be the purely equational calculus on $0, S, I$, with ordinary equality rules, the two iterator equations, and literal-equation induction with parameters. In particular, $\mathcal{E}$ has no axiom saying that every number is zero or a successor. The following strengthens the dependency statement of the inherited scalar predecessor construction:

$$\mathcal{E} \vdash P(0) = 0, \qquad \mathcal{E} \vdash P(Sx) = x$$

for one finite iterator term P. Thus $\mathcal{E}$ proves successor injectivity. The construction below is inherited; the new point is the explicit elimination of all its numerical and Boolean case splits.

Write $1 = S0$ and define

$$C(e, a, b) = I(e, a; u.b), \qquad Z(e) = C(e, 1, 0), \qquad a + b = I(b, a; u.Su).$$

The equations of a commutative monoid for addition, and $Sa + b = S(a + b)$, follow by equation induction. Expressions such as $2a$ and $3a$ abbreviate repeated addition.

Equational interpolation. For every term template $F(v)$, with all other variables fixed, equation induction on a fresh e proves

$$F(Z(e)) = C(e, F(1), F(0)). \tag{BI}$$

At $e = 0$ both sides are $F(1)$; at $e = Sz$ both sides are $F(0)$. The induction hypothesis is unused. Also equation induction, again with an unused hypothesis, proves $Z(Z(Z(e))) = Z(e)$. Consequently, if $b = Z(Z(b))$, and the two equations $F(0) = G(0)$ and $F(1) = G(1)$ are available under the current fixed parameters, then $F(b) = G(b)$ follows by applying (BI) at $e = Z(b)$. This is a derived equality rule, not a disjunction elimination rule. It continues to hold in the presence of arbitrary fixed side equations.

Parity and a three-cycle. Put

$$N(n) = I(n, 0; u.Z(u)), \quad j(u) = Z(u) + 2N(u), \quad M(n) = I(n, 0; u.j(u)).$$

Equation induction proves $N(n) = Z(Z(N(n)))$: the zero case computes; the successor case is $Z(N(n)) = Z(Z(Z(N(n))))$. Applying (BI) therefore gives

$$N(n) + Z(N(n)) = 1, \qquad N(n + 2) = N(n).$$

Equation inductions give

$$N(2n) = 0, \qquad N(2n + 1) = 1, \qquad N(3n) = N(n).$$

For the last equation use $N(a + 3) = Z(N(a))$ in the successor step. Finite computations yield $j(0) = 1, j(1) = 2, j(2) = 0$.

The ramp invariant, with its case split removed. Define

$$G(n) = I(n, 0; u.u + M(u) + 1).$$

We prove $M(G(n)) = N(n)$ by equation induction. At the successor step write $a = G(n)$ and $b = N(n)$, so the sole induction equation is $M(a) = b$. Interpolation in the Boolean parameter b gives

$$M(a + b + 1) = C(b, M(a + 1), M(a + 2)).$$

Both endpoint equations follow from the computations of C. Rewriting by $M(a) = b$ and the two computations of M turns the right side into $C(b, j(b), j(j(b)))$. A further use of (BI) gives

$$C(b, j(b), j(j(b))) = Z(b) :$$

at $b = 0$ both sides are 1, and at $b = 1$ both sides are 0. Since $G(Sn) = a + b + 1$ and $N(Sn) = Z(b)$, this proves the induction step. No disjunctive fact about b was used.

It follows by equality reasoning and equation induction that

$$G(Sn) = G(n) + N(n) + 1, \quad G(n + 2) = G(n) + 3, \quad G(2n) = 3n.$$

Another equation induction proves

$$G(3n) = 3G(n) + N(n).$$

Its successor step reduces, with $b = N(n)$, to the Boolean identity $2b + 4 = 3b + 3 + Z(b)$, whose endpoints are checked directly and whose general Boolean instance follows by (BI).

The factor map. Set $H(u) = C(N(u), G(u), u)$. We obtain

$$H(2u) = 3u, \qquad H(1) = 1, \qquad H(3u) = 3H(u).$$

Only the last identity requires comment. With $b = N(u)$ it is

$$C(b, 3G(u) + b, 3u) = 3C(b, G(u), u),$$

which is again (BI): its zero endpoint is $3G(u) = 3G(u)$ and its one endpoint is $3u = 3u$. The apparent dependence between b and u causes no problem: prove the displayed identity with an independent Boolean variable, then substitute $N(u)$.

Comparator. For any fixed update h, equation induction gives rotation $I(Sy, a; h) = I(y, h(a); h)$. Equation induction also gives

$$I(y, 3a; H) = 3I(y, a; H), \qquad I(y, 1; H) = 1.$$

Let

$$E(x) = I(x, 1; u.2u), \qquad L(x, y) = N(I(y, E(x); H)).$$

The preceding identities give the complete comparator specification

$$L(0, y) = 1, \qquad L(Sx, 0) = 0, \qquad L(Sx, Sy) = L(x, y).$$

For the third equation the calculation is

$$\begin{aligned} N(I(Sy, 2E(x); H)) &= N(I(y, H(2E(x)); H)) \\ &= N(I(y, 3E(x); H)) \\ &= N(3I(y, E(x); H)) = N(I(y, E(x); H)). \end{aligned}$$

Saturation and predecessor. Put

$$B_m(u) = C(L(Su, m), u, Su), \qquad W(t, m) = I(t, 0; u.B_m(u)).$$

The identity $C(e, Sa, Sb) = SC(e, a, b)$ is an equation induction on e with unused induction hypothesis. It gives

$$B_{Sm}(Su) = SB_m(u), \qquad B_{Sm}(0) = S0.$$

Equation induction on t, with m fixed, gives $W(St, Sm) = SW(t, m)$. Equation inductions then give

$$L(Sn, n) = 0, \qquad W(n, n) = n, \qquad W(Sn, n) = n.$$

In the middle equation substitute $t = m = n$ into the already established two-parameter shift equation. No parameter of a live induction hypothesis is changed.

Finally define the finite scalar term

$$P(m) = I(m, 0; u.C(L(SSu, m), u, Su)).$$

Its zero value computes. The comparator recurrence identifies the update of $P(Sm)$ with $B_m(u)$. Formally, equation induction on a fresh counter t proves

$$I(t, 0; u.C(L(SSu, Sm), u, Su)) = W(t, m).$$

At $t = Sm$ the right side is m, proving $P(Sm) = m$. This last induction accounts for iterator-body extensionality; it is not an extra equality axiom. Applying P to $Sx = Sy$ proves $x = y$.

Audit boundary. The proof uses only the two iterator equations, equality reasoning, substitution into established equations, and equation induction with fixed side parameters. Every Boolean step has been replaced by (BI). It does not use coverage, a predecessor axiom, successor injectivity, equality reflection, pairing, or induction on a non-equational formula. This is a written syntactic derivation; it is not a new Lean verification.

B Reproduction and provenance

From this directory run

```
python3 reproduce.py
python3 code/checker.py < code/examples/accepted/universal_coverage_closure.json
pdflatex -no-shell-escape -halt-on-error paper.tex
pdflatex -no-shell-escape -halt-on-error paper.tex
```

The exact example inventory and test commands are recorded in `README.txt`. The package includes the frozen incoming checker, the new checker, normalized sources, measurements, validation scripts, the mathematical notes, and the inherited bridge sources used in the proof. `PROVENANCE.json` identifies the repository snapshot and the five incoming archives; `SHA256SUMS` covers the delivered payload. The upstream material is distributed under its MIT No Attribution license, preserved in `LICENSE-upstream.txt`.

References

[1] ProveIt contributors. *Concise presentations of intuitionistic primitive recursive arithmetic.* Snapshot `9f9545f5ffd9e0392d4a36639503aca12383450a`, November 12, 2026. https://github.com/VladimirReshetnikov/ProveIt/tree/9f9545f5ffd9e0392d4a36639503aca12383450a/Logic/PrimitiveRecursiveArithmetic/ConciseIPRA.

[2] *Deterministic introduction for intuitionistic primitive recursive arithmetic.* Incoming archive `ipra_deterministic_introduction.zip`, in the snapshot above. Its strict checker is the matched control in this package.

[3] *iPRA without supplied introduction witnesses.* Incoming archive `ipra_parameter_intro_2026-11-11.zip`, in the snapshot above.

[4] *iPRA with evaluation at witness discharge.* Incoming archive `ipra_evaluation_discharge_2026-11-11.zip`, in the snapshot above.

[5] ProveIt, `Research/guarded_core` and its `guarded_core.tex` bridge, also reproduced in the incoming deterministic-introduction archive. Its canonical formula grammar, padding, equation implication, and guarded/native translations are inherited here.

[6] ProveIt, `Research/certified_zero/reference`: `inherited_choice_basis.tex`, `inherited_ordinary_bridge.tex`, `scalar_predecessor.tex`, `equality_reflection.tex`, and `primitive_recursion.tex`. Copies are included under `reference/`.

[7] ProveIt, `Lean/ConciseIPRA`, especially `NativePublic.lean`, `NativePublicD.lean`, `NativeBiInterpretation.lean`, and `BiInterpretation.lean`. The snapshot records the established native/scalar/iPRA equivalences; it does not formalize the new restriction proved here.